Audio Signal Extraction

Capturing and normalizing classroom audio for multimodal analysis

Assignee	Work Item	Allocated Duration	Status
Humesh Sharma	Story	1h 30m	In Progress

1. Overview & Scope

In this activity, audio input was captured from a **microphone** or extracted from **recorded video files** associated with classroom sessions. The raw audio signal was processed into a digital waveform suitable for analysis. **Sampling rate normalization** and **channel standardization** were applied to ensure consistent processing across different audio sources.

The extracted signal was segmented into **fixed-duration windows** to enable time-based analysis aligned with video. These segments form the base input for noise and speech activity detection in subsequent stages. Processed audio data was stored in structured formats, including timestamps and amplitude information, to facilitate synchronized multimodal analysis.

Extraction Pipeline

1. Capture
Microphone input or audio extracted from recorded video

→

2. Normalize
16 kHz sampling, mono channel, float32 range

→

3. Segment
Fixed-duration time windows

→

4. Store
Timestamps + amplitude in structured form

2. Technical Specifications & Ingestion Parameters

Parameter	Configured Value	Design Justification
Sampling Rate	16,000 Hz (16 kHz)	Standard frequency for acoustic speech models and voice activity detection (VAD).
Channel Format	Single channel (mono)	Eliminates phase discrepancies and standardizes multi-mic classroom setups.
Quantization Depth	32-bit float ([-1.0, 1.0])	Normalized from raw 16-bit PCM integer buffers to avoid numerical clipping during FFT.
Windowing Alignment	Fixed time windows	Enables precise timestamp correlation with facial engagement and attendance video frames.

Table 1: Ingestion parameters and the rationale behind each.

3. Module Implementation

The module is `backend/core/audio_analysis.py` (Python 3.10+). It defines an `AudioAnalyzer` class configured with a default sample rate of 16,000 Hz. Its analysis method accepts raw audio as either **bytes** or an **array**, and is documented to compute three indicators and return engagement metrics:

Indicator	Description
Noise level	Ambient sound level expressed in decibels.
Speech vs silence ratio	Proportion of the signal containing speech versus silence.
Sound activity patterns	How sound activity is distributed over the analyzed audio.

Input Handling

When the input is `bytes`, it is interpreted as 16-bit PCM, converted to `float32` and divided by 32768 so values fall in [-1.0, 1.0]. Array input is used as provided. This is the step that implements the *Quantization Depth* specification above.

Code Skeleton (legible portions of the source)

Only the parts that are legible in the source file are shown. `...` marks content that could not be recovered.

```
# File: backend/core/audio_analysis.py   (Python 3.10+)
import librosa
import numpy as np
import os

class AudioAnalyzer:
    def __init__(self, sample_rate=16000):
        self.sample_rate = sample_rate

    def analyze_...(self, audio_data):
        """
        Analyzes raw audio bytes/array:
        1. Noise level (decibels)
        2. Speech vs silence ratio
        3. Sound activity patterns

        Returns engagement metrics
        """
        try:
            # Convert bytes to float32 numpy array if needed
            if isinstance(audio_data, bytes):
                # Assuming 16-bit PCM
                y = np.frombuffer(audio_data, dtype=np.int16).astype(np.float32) / 32768
            else:
                y = audio_data

            if len(y) == ...:
                ...    # empty-input handling

            ...        # metric computation

            return {..., "...": len(y), "...": self.sample_rate}
        except Exception as e:
            return {"...": str(e)}
```

4. Downstream Multimodal Pipeline Integration

Synchronous Alignment Note. Audio chunks extracted in this unit pass through normalization and are correlated via ISO timestamps with concurrent frame outputs from the visual attendance tracker, driving integrated attention-index scoring across classroom sessions.

Documentation Notes

Source code partly unreadable. The code block in the original Activity 3.1 file is corrupted: many keywords, string literals, the empty-input branch, the metric computation, and the return keys are missing or scrambled. The skeleton above therefore shows only legible content, and the method name is truncated in the source (`analyze_...`). Re-exporting the original code block would allow a complete listing.

Window length not stated. The source specifies fixed-duration windows but not their length, and the code shown does not perform the segmentation step itself.